06
Design A Rate Limiter

In a network system, a rate limiter is used to control the rate of traffic sent by a client or a service. In the HTTP world, a rate limiter limits the number of client requests allowed to be sent over a specified period. If the count of the requests is reached, the additional requests are blocked.

Rate limiting can be implemented using different algorithms, each with its pros and cons. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Step 1 - Understand the problem and establish design scope

Rate limiting can be implemented using different algorithms, each with its pros and cons. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Assume we have a RESTful API service, and the API is used to block users based on their IP addresses. Assume an API client is allowed to send 2 client requests to the API every second. If the client requests are exceeding the API service, however, the rate limiter middleware throttles the third request and returns a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rate limited.

Figure 1

Client implementation: An HTTP client is a client-side program that handles requests to the server. Usually, the client is a browser, a mobile app, or a JavaScript client. We can assume that the client request is properly formatted, and the request is not forged.

Rate limiter: Rate limiting can be implemented on the server side or in a middleware. In our design, we choose to put the rate limiter in a middleware that throttles requests to the server.

Web server: The web server is a server that serves web content. We can assume that the web server is a server that serves web content. We can assume that the web server is a server that serves web content.

Rate limiting can be implemented using different algorithms, each with its pros and cons. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 2

Step 2 - Propose high-level design and get buy-in

Let us take things step-by-step and discuss each component in detail.

Where to put the rate limiter?

Rate limiting can be implemented using different algorithms, each with its pros and cons. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Figure 3

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 4

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 5

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 6

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 7

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 8

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 9

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 10

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 11

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 12

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 13

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 14

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 15

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 16

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 17

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 18

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 19

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 20

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 21

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 22

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 23

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 24

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 25

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 26

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 27

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 28

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 29

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 30

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 31

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 32

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 33

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 34

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 35

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 36

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 37

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 38

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 39

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 40

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 41

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 42

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 43

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 44

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 45

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 46

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 47

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 48

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 49

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 50

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 51

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 52

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 53

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 54

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 55

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 56

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 57

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 58

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 59

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 60

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 61

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 62

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 63

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 64

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 65

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 66

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 67

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 68

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 69

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 70

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 71

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 72

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 73

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 74

Rate limiting can be implemented using different algorithms, each with its pros and cross. The interactions between an rate limiter and its clients are also fairly standard, as defined by HTTP protocol. This chapter discusses many popular rate limiting algorithms and how they are implemented in a distributed environment.

Rate limiting can be implemented at various places in the network. For example, it can be implemented on the client's side, on the server's side, or in a middleware that routes the client request and enforces a HTTP status code 429. The HTTP 429 response status code indicates that a client's request is rejected.

Figure 75

Rate limi

Synchronization issue

Synchronization is another important factor to consider in a distributed environment. To support millions of users, one rate limiter server might not be enough to handle the traffic. When multiple rate limiter servers are used, synchronization is required. For example, on the left side of Figure 15, client 1 sends requests to rate limiter 1, and client 2 sends requests to rate limiter 2. As the web tier is stateless, clients can send requests to a different rate limiter as shown on the right side of Figure 15. If no synchronization happens, rate limiter 1 does not contain any data about client 2. Thus, the rate limiter cannot work properly.

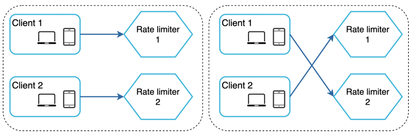

Figure 15

One possible solution is to use sticky sessions that allow a client to send traffic to the same rate limiter. This solution is not advisable because it is neither scalable nor flexible. A better approach is to use centralized data stores like Redis. The design is shown in Figure 16.

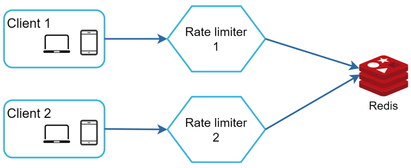

Figure 16

Performance optimization

Performance optimization is a common topic in system design interviews. We will cover two areas to improve.

First, multi-data center setup is crucial for a rate limiter because latency is high for users located far away from the data center. Most cloud service providers build many edge server locations around the world. For example, as of 5/20 2020, Cloudflare has 194 geographically distributed edge servers [14]. Traffic is automatically routed to the closest edge server to reduce latency.

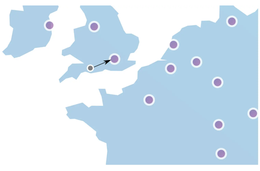

Figure 17 (Source: [10])

Second, synchronize data with an eventual consistency model. If you are unclear about the eventual consistency model, refer to the "Consistency" section in the "Design a Key-value Store" chapter.

Monitoring

After the rate limiter is put in place, it is important to gather analytics data to check whether the rate limiter is effective. Primarily, we want to make sure:

- The rate limiting algorithm is effective.
- The rate limiting rules are effective.

For example, if rate limiting rules are too strict, many valid requests are dropped. In this case, we want to relax the rules a little bit. In another example, we notice our rate limiter becomes ineffective when there is a sudden increase in traffic like flash sales. In this scenario, we may replace the algorithm to support burst traffic. Token bucket is a good fit here.

Step 4 - Wrap up

In this chapter, we discussed different algorithms of rate limiting and their pros/cons. Algorithms discussed include:

- Token bucket
- Leaking bucket
- Fixed window
- Sliding window log
- Sliding window counter

Then, we discussed the system architecture, rate limiter in a distributed environment, performance optimization and monitoring. Similar to any system design interview questions, there are additional talking points you can mention if time allows:

- Hard vs soft rate limiting.
- Hard: The number of requests cannot exceed the threshold.
- Soft: Requests can exceed the threshold for a short period.
- Rate limiting at different levels. In this chapter, we only talked about rate limiting at the application level (HTTP: layer 7). It is possible to apply rate limiting at other layers. For example, you can apply rate limiting by IP addresses using Iptables [15] (IP: layer 3). Note: The Open Systems Interconnection model (OSI model) has 7 layers [16]: Layer 1: Physical layer, Layer 2: Data link layer, Layer 3: Network layer, Layer 4: Transport layer, Layer 5: Session layer, Layer 6: Presentation layer, Layer 7: Application layer.
- Avoid being rate limited. Design your client with best practices:
- Use client cache to avoid making frequent API calls.
- Understand the limit and do not send too many requests in a short time frame.
- Include code to catch exceptions or errors so your client can gracefully recover from exceptions.
- Add sufficient back off time to retry logic.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Reference materials

[1] Rate-limiting strategies and techniques:
https://cloud.google.com/solutions/rate-limiting-strategies-techniques

[2] Twitter rate limits: https://developer.twitter.com/en/docs/basics/rate-limits

[3] Google docs usage limits: https://developers.google.com/docs/api/limits

[4] IBM microservices: https://www.ibm.com/cloud/learn/microservices

[5] Throttle API requests for better throughput:
https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-request-throttling.html

[6] Stripe rate limiters: https://stripe.com/blog/rate-limiters

[7] Shopify REST Admin API rate limits:
https://help.shopify.com/en/api/reference/rest-admin-api-rate-limits

[8] Better Rate Limiting With Redis Sorted Sets:
https://engineering.classdojo.com/blog/2015/02/06/rolling-rate-limiter/

[9] System Design — Rate limiter and Data modelling:
https://medium.com/@saisandeepmopuri/system-design-rate-limiter-and-data-modelling-9304b0d18250

[10] How we built rate limiting capable of scaling to millions of domains:
https://blog.cloudflare.com/counting-things-a-lot-of-different-things/

[11] Redis website: https://redis.io/

[12] Lyft rate limiting: https://github.com/lyft/ratelimit

[13] Scaling your API with rate limiters:
https://gist.github.com/ptarjan/e38f45f2dfe601419ca3af937fff574d#request-rate-limiter

[14] What is edge computing:
https://www.cloudflare.com/learning/serverless/glossary/what-is-edge-computing/

[15] Rate Limit Requests with Iptables: https://blog.programster.org/rate-limit-requests-with-iptables

[16] OSI model:
https://en.wikipedia.org/wiki/OSI_model#Layer_architecture

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.